

import try/except

Módulos y Manejo de Errores en Python

Divide tu código y controla lo inesperado

De PSeInt a Python — Guía Práctica Paso a Paso

Docente: Jhonathan Arley Peñaloza Sierra

Skill: Introducción a Python | Campuslands · 2026

[import](#) · [from](#) · [alias](#) · [iter](#) · [try](#) · [except](#) · [else](#) · [finally](#)

Antes de empezar: lee esto primero

Este tema tiene dos partes independientes. Puedes estudiarlas en cualquier orden, pero te recomendamos ir de módulos a errores porque los ejercicios avanzados las combinan.

Parte 1 — Módulos: implica MÚLTIPLES archivos .py

A diferencia de todo lo anterior, los ejercicios de módulos requieren crear más de un archivo .py y que estén en la misma carpeta (o en subcarpetas específicas). Cada ejercicio indica exactamente qué archivo crear y qué código va en cada uno.

Regla clave: el archivo que tiene el `import` debe estar en la MISMA carpeta que el módulo que importa.

Parte 2 — Manejo de Errores: todo en un solo archivo

Los ejercicios de try/except funcionan en un solo archivo .py. No necesitas crear ninguna estructura especial. Solo escribes, ejecutas y observas cómo Python maneja (o no maneja) los errores.

Conceptos Clave

Repasa esta sección cuando tengas dudas durante los ejercicios.

Formas de importar módulos

Forma	Sintaxis	Cuándo usarla
Importar todo el módulo	<code>import mi_modulo</code>	Cuando quieres tener claro de dónde viene cada función.
Importar una función	<code>from mi_modulo import suma</code>	Cuando solo necesitas una o pocas funciones específicas.
Importar todo sin prefijo	<code>from mi_modulo import *</code>	Cuando quieres usar todas las funciones sin escribir el nombre del módulo.
Alias (renombrar)	<code>import mi_modulo as mm</code>	Cuando el nombre es largo o entra en conflicto con otro módulo.

 **Recomendación para principiantes:** empieza con `from mi_modulo import *`. Es la más simple. Cuando tengas más experiencia, prefiere importar solo lo que necesitas.

Estructura completa de manejo de errores

```
try:
    # Código que PUEDE fallar
    resultado = int(input('Ingresa un número: ')) / int(input('Dividelo entre: '))

except ValueError:
    # Si el usuario escribe texto en vez de número
    print('Error: debes ingresar un número válido.')

except ZeroDivisionError:
    # Si el usuario divide entre cero
    print('Error: no se puede dividir entre cero.')

except Exception as e:
    # Cualquier otro error no previsto
    print(f'Error inesperado: {e}')

else:
    # Se ejecuta SOLO si no hubo ningún error
    print(f'Resultado: {resultado}')

finally:
    # Se ejecuta SIEMPRE, haya error o no
    print('Operación finalizada.')
```

Excepciones más comunes

Excepción	Cuándo ocurre	Ejemplo que la provoca
TypeError	Tipo de dato incorrecto	5 + "2"
ValueError	Valor no convertible	int("abc")
ZeroDivisionError	División entre cero	10 / 0
IndexError	Índice fuera de rango	lista[99] (lista de 3 elementos)
KeyError	Key inexistente en diccionario	dic["no_existe"]
FileNotFoundError	Archivo que no existe	open("fantasma.txt","r")
AttributeError	Método que no existe en el tipo	(5).upper()

💡 **Tip para leer Tracebacks:** léelo de abajo hacia arriba. La última línea dice exactamente qué tipo de error es y en qué línea ocurrió. No le tengas miedo — es tu mejor herramienta de diagnóstico.

PARTE 1: EJERCICIOS GUIADOS

Sigue cada paso como si estuvieras con un tutor a tu lado

Tema 1: Módulos — Dividir tu programa en archivos

La modularidad es el principio de dividir un programa grande en archivos .py más pequeños, cada uno con una responsabilidad específica. En Python, cada archivo .py es un módulo. Importar ese módulo es traer su código a tu programa principal.

Ejercicio 1.1 — Tu primer módulo: calculadora

Enunciado

Crea dos archivos: calculadora.py con las operaciones matemáticas básicas, y main.py que importe y use esas operaciones. Practica las tres formas de importar.

Guía paso a paso:

Estructura de archivos para este ejercicio:

```
mi_proyecto/  
├── calculadora.py  ← el módulo (las funciones)  
└── main.py         ← el programa principal (importa el módulo)
```

Ambos archivos deben estar en la MISMA carpeta. Crea la carpeta, luego crea los dos archivos dentro.

Paso 1: Crea el módulo calculadora.py.

 `calculadora.py`

```
"""  
Módulo de operaciones matemáticas básicas.  
Este archivo es el MÓDULO — no se ejecuta directamente.  
"""  
  
def sumar(a, b):  
    return a + b  
  
def restar(a, b):  
    return a - b  
  
def multiplicar(a, b):  
    return a * b  
  
def dividir(a, b):  
    if b == 0:  
        return 'Error: no se puede dividir entre cero'  
    return a / b
```

Este archivo SOLO tiene funciones. No tiene input(), no tiene print() sueltos, no tiene código que se ejecute al importarlo. Eso es una buena práctica.

Paso 2: Crea main.py e importa de tres formas distintas.

main.py

```
# — Forma 1: importar el módulo completo —————
import calculadora

print(calculadora.sumar(5, 3))      # → 8
print(calculadora.multiplicar(4, 7)) # → 28

# — Forma 2: importar solo lo que necesitas —————
from calculadora import sumar, dividir

print(sumar(10, 20))    # → 30 (sin prefijo 'calculadora.')
print(dividir(15, 3))   # → 5.0

# — Forma 3: alias —————
import calculadora as calc

print(calc.restar(100, 45)) # → 55 (prefijo corto 'calc.')
```

Ejecuta main.py (no calculadora.py). Si Python encuentra las funciones, el import funcionó correctamente.

Error más común en módulos:

```
# ModuleNotFoundError: No module named 'calculadora'

# Causa: main.py y calculadora.py no están en la misma carpeta.
# Solución: verifica con este código al inicio de main.py:
import os
print(os.getcwd()) # muestra en qué carpeta está Python buscando
```

Mini Reto 1.1

- Agrega a calculadora.py una función potencia(base, exponente) que retorne base ** exponente.
- Agrega también modulo(a, b) que retorne el residuo de la división (a % b).
- Úsalas en main.py con las tres formas de importar.

Solución sugerida — Ejercicio 1.1

calculadora.py

```
def sumar(a, b):      return a + b
def restar(a, b):     return a - b
def multiplicar(a,b): return a * b
def dividir(a, b):    return a / b if b != 0 else 'Error: división por cero'
def potencia(b, e):   return b ** e
def modulo(a, b):     return a % b
```

main.py

```
import calculadora
from calculadora import potencia, modulo
import calculadora as calc

print(calculadora.sumar(5, 3))    # 8
print(potencia(2, 10))            # 1024
print(modulo(17, 5))              # 2
print(calc.dividir(22, 7))        # 3.142857...
```

Explicación:

- El módulo es solo funciones — sin código suelto. El programa principal (main.py) es quien ejecuta.
- `import calculadora` → usas `calculadora.sumar()` | `from calculadora import sumar` → usas `sumar()` directo.
- El alias (`import calculadora as calc`) es más útil cuando el nombre del módulo es largo.

Ejercicio 1.2 — Módulo con subcarpeta: tienda online

Enunciado

Crea una estructura de proyecto más realista: el programa principal en la raíz, y los módulos organizados en una subcarpeta. Separa la lógica del menú en un módulo y las operaciones de productos en otro.

Guía paso a paso:

Estructura de archivos para este ejercicio:

```
tiendaOnline/
├── main.py           ← programa principal
├── modulos/         ← carpeta con los módulos
│   ├── __init__.py  ← archivo vacío (marca la carpeta como paquete)
│   ├── menu.py      ← módulo del menú
│   └── productos.py ← módulo de productos
```

El archivo `__init__.py` debe existir (puede estar vacío) para que Python reconozca la carpeta como un paquete importable.

Paso 1: Crea el módulo menu.py.

 `modulos/menu.py`

```
def mostrar_menu(opciones):
    print("\n— Menú —")
    for i, opcion in enumerate(opciones, 1):
        print(f"{i}. {opcion}")
    seleccion = input("Elige una opción: ")
    return seleccion
```

Paso 2: Crea el módulo productos.py.

 `modulos/productos.py`

```
catalogo = {
    "Menta": 300,
    "Chocorrano": 1000,
    "Esfero": 2700,
}

def mostrar_catalogo():
    print("\n— Catálogo —")
    for producto, precio in catalogo.items():
        print(f" {producto}: ${precio}")
```

```
def calcular_total(producto, cantidad):
    if producto not in catalogo:
        return None
    return catalogo[producto] * cantidad
```

Paso 3: Crea `__init__.py` (vacío).

 `modulos/__init__.py`

```
# Este archivo puede estar vacío.
# Su presencia le dice a Python que 'modulos' es un paquete.
```

Paso 4: Crea `main.py` que orquesta todo.

 `main.py`

```
from modulos.menu import mostrar_menu
from modulos.productos import mostrar_catalogo, calcular_total

opciones = ['Ver catálogo', 'Comprar producto', 'Salir']

while True:
    sel = mostrar_menu(opciones)
    if sel == '1':
        mostrar_catalogo()
    elif sel == '2':
        mostrar_catalogo()
        producto = input('¿Qué producto? ').title()
        cantidad = int(input('¿Cuántos? '))
        total = calcular_total(producto, cantidad)
        if total is None:
            print('Producto no encontrado.')
        else:
            print(f'Total: ${total}')
    elif sel == '3':
        print('Hasta luego!')
        break
    else:
        print('Opción inválida.')
```

Mini Reto 1.2

- Agrega a `productos.py` una función `agregar_producto(nombre, precio)` que añada el producto al catálogo.
- Agrégala al menú como opción 3 (desplaza 'Salir' a opción 4).

Pista: `catalogo[nombre] = precio` dentro de la función.

Solución sugerida — Ejercicio 1.2

 `modulos/productos.py` (solo la función nueva)

```
def agregar_producto(nombre, precio):
    catalogo[nombre] = precio
    print(f"{nombre} agregado al catálogo.")
```

 `main.py` (fragmento con opción nueva)

```
from modulos.productos import mostrar_catalogo, calcular_total, agregar_producto

opciones = ['Ver catálogo', 'Comprar', 'Agregar producto', 'Salir']

# dentro del while:
elif sel == '3':
    nombre = input('Nombre del producto: ').title()
    precio = int(input('Precio: '))
```



```
agregar_producto(nombre, precio)
```

Explicación:

- from modulos.menu import ... usa puntos para navegar subcarpetas: modulos → menu.
- El __init__.py vacío es obligatorio para que Python trate la carpeta como paquete.
- Separar menú y productos en módulos distintos es el principio de responsabilidad única: cada archivo tiene una sola razón para cambiar.

Ejercicio 1.3 — Iterables: iter() y next()

Enunciado

Aprende a recorrer colecciones con iter() y next() en lugar de for. Este mecanismo te da control total: avanzas uno a uno, puedes pausar, saltar o reaccionar ante el fin de la colección.

Guía paso a paso:

Paso 1: iter() y next() básicos.

```
repisa = ['Peluche', 'Libro', 'Celular', 'Lámpara']

# Crear el iterador
objetos = iter(repisa)

# Avanzar uno por uno con next()
print(next(objetos)) # → Peluche
print(next(objetos)) # → Libro
print(next(objetos)) # → Celular
print(next(objetos)) # → Lámpara

# Si llamas next() una vez más:
# print(next(objetos)) → StopIteration (no hay más elementos)
```

iter() convierte la lista en un iterador. next() extrae el siguiente elemento cada vez que lo llamas. El iterador recuerda en qué posición está.

Paso 2: Manejar el fin del iterador con StopIteration.

```
nombres = ['Ana', 'Luis', 'María']
it = iter(nombres)

while True:
    try:
        nombre = next(it)
        print(f"Procesando: {nombre}")
    except StopIteration:
        print("No hay más nombres.")
        break

# Salida:
# Procesando: Ana
# Procesando: Luis
# Procesando: María
# No hay más nombres.
```

StopIteration es la excepción que Python lanza cuando un iterador se queda sin elementos. Capturarla con try/except es la forma correcta de saber cuándo terminar.

Paso 3: Iterar una cadena de texto carácter por carácter.

```
palabra = 'Python'
letras = iter(palabra)

for _ in range(3): # lee solo las primeras 3 letras
    print(next(letras))

# → P
# → y
# → t

# El iterador recuerda dónde quedó:
print(next(letras)) # → h (continúa desde donde se pausó)
```

A diferencia de un for, el iterador mantiene su estado. Puedes pausarlo y retomarlo después, o decidir cuántos elementos procesar en cada momento.

Mini Reto 1.3

- Crea un iterador sobre una lista de números. Usa next() dentro de un while True y try/except para sumar todos los números sin usar for.
- Crea un iterador sobre las keys de un diccionario con iter(diccionario.keys()) y recórrelo con next().

Pista: el try/except StopIteration reemplaza el for cuando necesitas control manual.

Solución sugerida — Ejercicio 1.3

```
# Mini reto 1: sumar con iterador
numeros = [10, 25, 7, 43, 8]
it = iter(numeros)
total = 0
while True:
    try:
        total += next(it)
    except StopIteration:
        break
print(f"Suma: {total}") # → 93

# Mini reto 2: iterar keys de diccionario
persona = {'nombre': 'Ana', 'edad': 25, 'ciudad': 'Cúcuta'}
it_keys = iter(persona.keys())
while True:
    try:
        key = next(it_keys)
        print(f'{key}: {persona[key]}')
    except StopIteration:
        break
```

Explicación:

- iter() + next() + StopIteration es la base interna de cómo funciona for en Python.
- Usar next() manualmente tiene sentido cuando quieres procesar elementos de forma no lineal o pausada.
- Cualquier iterable (lista, tupla, cadena, diccionario, conjunto) puede convertirse en iterador con iter().

Tema 2: Manejo de Errores — try / except / else / finally

Sin manejo de errores, un solo input incorrecto del usuario puede detener todo tu programa. Con try/except, capturas lo inesperado y decides qué hacer. Esto es lo que separa un programa frágil de uno robusto.

Ejercicio 2.1 — Calculadora robusta: nunca se detiene

Enunciado

Crea una calculadora que pida dos números y una operación. El programa NO debe detenerse si el usuario ingresa texto en lugar de números, ni si intenta dividir entre cero. Debe seguir pidiendo datos hasta obtener una entrada válida.

Guía paso a paso:

Paso 1: Captura un solo tipo de error.

```
# Sin try/except — el programa MUERE si el usuario escribe 'abc':
n = int(input('Ingresa un número: ')) # ValueError si no es número

# CON try/except — el programa SOBREVIVE:
try:
    n = int(input('Ingresa un número: '))
    print(f'Número recibido: {n}')
except ValueError:
    print('Eso no es un número. Intenta de nuevo.')
```

El bloque try intenta ejecutar el código. Si ocurre un ValueError (el usuario escribió texto), Python salta al except correspondiente en lugar de detener el programa.

Paso 2: Captura múltiples errores con un bloque por tipo.

```
try:
    n1 = int(input('Primer número: '))
    n2 = int(input('Segundo número: '))
    resultado = n1 / n2

except ValueError:
    print('Error: debes ingresar solo números enteros.')

except ZeroDivisionError:
    print('Error: no puedes dividir entre cero.')

except Exception as e:
    print(f'Error inesperado: {e}')

else:
    print(f'Resultado: {resultado}')

finally:
    print('Operación finalizada.')
```

Flujo de ejecución — aprende a leerlo:

try	Siempre se intenta primero
except	Solo si ocurrió el error especificado
else	Solo si NO ocurrió ningún error
finally	SIEMPRE, ocurra lo que ocurra

Paso 3: Forzar al usuario a dar una entrada válida con while.

```
def pedir_numero(mensaje):
    while True:
        try:
            return int(input(mensaje))
        except ValueError:
            print(' ⚠ Escribe solo números enteros.')

n1 = pedir_numero('Primer número: ')
n2 = pedir_numero('Segundo número: ')

try:
    resultado = n1 / n2
except ZeroDivisionError:
    print('No se puede dividir entre cero.')
else:
    print(f'{n1} / {n2} = {resultado:.2f}')
```

Envolver el try/except dentro de while True y retornar con return cuando el dato es válido es el patrón estándar para validación robusta de entradas.

Mini Reto 2.1

- Convierte la calculadora en un menú completo (sumar, restar, multiplicar, dividir) que repite hasta que el usuario elija salir.
- Cada operación debe manejar sus errores correspondientes.

Pista: combina el módulo calculadora.py del Tema 1 con el manejo de errores de este ejercicio.

Solución sugerida — Ejercicio 2.1

```
def pedir_numero(mensaje):
    while True:
        try:
            return float(input(mensaje))
        except ValueError:
            print(' ⚠ Ingresa un número válido.')

operaciones =
{'1': 'Sumar', '2': 'Restar', '3': 'Multiplicar', '4': 'Dividir', '5': 'Salir'}

while True:
    print('\n— Calculadora —')
    for k, v in operaciones.items(): print(f' {k}. {v}')
    sel = input('Opción: ')

    if sel == '5': print('Hasta luego!'); break
    if sel not in operaciones:
        print('Opción inválida.');
```

```

a = pedir_numero('Primer número: ')
b = pedir_numero('Segundo número: ')

try:
    if sel=='1': r = a + b
    elif sel=='2': r = a - b
    elif sel=='3': r = a * b
    elif sel=='4': r = a / b
except ZeroDivisionError:
    print('Error: división entre cero.')
else:
    print(f'Resultado: {r}')
finally:
    print('-' * 20)

```

Explicación:

- pedir_numero() repite hasta que el dato es válido — el usuario no puede romper el programa.
- El else después del try garantiza que el resultado solo se muestra si no hubo error.
- El finally muestra el separador siempre, haya o no error — útil para UI consistente.

Ejercicio 2.2 — Validador de datos del usuario

Enunciado

Crea un formulario de registro que pida nombre, edad y correo. Cada campo debe validarse con try/except. La edad debe ser un número entre 1 y 120. El correo debe contener '@'. Si algún dato es inválido, pide ese campo de nuevo.

Guía paso a paso:

Paso 1: Validar la edad con ValueError y rango.

```

def pedir_edad():
    while True:
        try:
            edad = int(input('Edad: '))
            if not (1 <= edad <= 120):
                raise ValueError('Edad fuera de rango')
            return edad
        except ValueError as e:
            print(f'⚠ Edad inválida: {e}')

```

raise ValueError(...) lanza manualmente una excepción. Esto te permite crear tus propias validaciones y manejarlas exactamente igual que los errores de Python.

Paso 2: Validar el correo con un raise personalizado.

```

def pedir_correo():
    while True:
        correo = input('Correo: ').strip()
        try:
            if '@' not in correo:
                raise ValueError('El correo debe contener @')

```

```

        if '.' not in correo.split('@')[1]:
            raise ValueError('El correo debe tener dominio válido')
        return correo
    except ValueError as e:
        print(f' ⚠ {e}')

```

Paso 3: Validar el nombre y ensamblar el formulario.

```

def pedir_nombre():
    while True:
        nombre = input('Nombre: ').strip()
        try:
            if len(nombre) < 2:
                raise ValueError('El nombre debe tener al menos 2 caracteres')
            if not nombre.replace(' ', '').isalpha():
                raise ValueError('El nombre solo debe contener letras')
            return nombre.title()
        except ValueError as e:
            print(f' ⚠ {e}')

# Formulario principal
print("—— Registro de usuario ——")
nombre = pedir_nombre()
edad = pedir_edad()
correo = pedir_correo()

print(f"\n✅ Usuario registrado:")
print(f"    Nombre: {nombre}")
print(f"    Edad:    {edad}")
print(f"    Correo:  {correo}")

```

Mini Reto 2.2

- Agrega una función pedir_telefono() que valide que el teléfono tenga exactamente 10 dígitos.
- Guarda los datos del usuario en un diccionario y muéstralos al final con un for.

Pista: raise ValueError('El teléfono debe tener 10 dígitos') si len(tel) != 10 or not tel.isdigit().

Solución sugerida — Ejercicio 2.2

```

def pedir_nombre():
    while True:
        try:
            n = input('Nombre: ').strip()
            if len(n)<2: raise ValueError('Mínimo 2 caracteres')
            if not n.replace(' ','').isalpha(): raise ValueError('Solo letras')
            return n.title()
        except ValueError as e: print(f' ⚠ {e}')

def pedir_edad():
    while True:
        try:
            e = int(input('Edad: '))
            if not 1<=e<=120: raise ValueError('Entre 1 y 120')
            return e
        except ValueError as e: print(f' ⚠ {e}')

def pedir_correo():
    while True:
        try:
            c = input('Correo: ').strip()
            if '@' not in c: raise ValueError('Debe contener @')

```

```

        return c
    except ValueError as e: print(f' ⚠ {e}')

def pedir_telefono():
    while True:
        try:
            t = input('Teléfono (10 dígitos): ').strip()
            if not t.isdigit() or len(t)!=10:
                raise ValueError('Debe tener exactamente 10 dígitos')
            return t
        except ValueError as e: print(f' ⚠ {e}')

usuario = {
    'nombre': pedir_nombre(),
    'edad': pedir_edad(),
    'correo': pedir_correo(),
    'telefono': pedir_telefono()
}

print('\n✅ Usuario registrado:')
for campo, valor in usuario.items():
    print(f' {campo.capitalize()}: {valor}')

```

Explicación:

- `raise ValueError(...)` te permite usar el mismo mecanismo de excepciones para tus propias reglas de negocio.
- `except ValueError as e` guarda el mensaje del error en `e`, para mostrarlo al usuario.
- Separar cada campo en su propia función hace el código legible y fácil de modificar.

Ejercicio 2.3 — Manejo de errores con archivos y módulos

Enunciado

Combina módulos con manejo de errores: crea un módulo `archivo_util.py` con funciones para leer y escribir archivos de forma segura (con `try/except` integrado). El programa principal lo importa y trabaja con archivos sin preocuparse por los errores de bajo nivel.

Guía paso a paso:

Estructura de archivos:

```

proyecto/
├── archivo_util.py  ← módulo con funciones seguras de archivo
└── main.py         ← programa principal

```

Paso 1: Crea el módulo `archivo_util.py` con funciones robustas.

`archivo_util.py`

```

def leer_archivo(ruta):
    """Intenta leer un archivo. Retorna el contenido o None si falla."""
    try:
        with open(ruta, 'r', encoding='utf-8') as f:

```



```

        return f.read()
    except FileNotFoundError:
        print(f' ⚠ Archivo no encontrado: {ruta}')
        return None
    except PermissionError:
        print(f' ⚠ Sin permisos para leer: {ruta}')
        return None

def escribir_archivo(ruta, contenido, modo='w'):
    """Escribe en un archivo. Retorna True si tuvo éxito."""
    try:
        with open(ruta, modo, encoding='utf-8') as f:
            f.write(contenido)
        return True
    except Exception as e:
        print(f' ⚠ Error al escribir {ruta}: {e}')
        return False

def leer_lineas(ruta):
    """Retorna lista de líneas o lista vacía si el archivo no existe."""
    try:
        with open(ruta, 'r', encoding='utf-8') as f:
            return [l.strip() for l in f.readlines() if l.strip()]
    except FileNotFoundError:
        return []

```

Paso 2: Usa el módulo en main.py sin preocuparte por errores.



main.py

```

from archivo_util import leer_archivo, escribir_archivo, leer_lineas

# Escribir datos
exito = escribir_archivo('notas.txt', 'Nota 1: Python es genial\n')
if exito:
    print('Archivo creado correctamente.')

# Añadir más contenido
escribir_archivo('notas.txt', 'Nota 2: Los módulos organizan el código\n',
modo='a')

# Leer todo
contenido = leer_archivo('notas.txt')
if contenido:
    print('\nContenido del archivo:')
    print(contenido)

# Intentar leer archivo inexistente - no se detiene el programa
resultado = leer_archivo('fantasma.txt')
print(f'Resultado: {resultado}')    # → None

# Leer líneas como lista
lineas = leer_lineas('notas.txt')
print(f'El archivo tiene {len(lineas)} líneas:')
for i, l in enumerate(lineas, 1):
    print(f' {i}. {l}')

```

Mini Reto 2.3

- Agrega a archivo_util.py una función contar_palabras(ruta) que lea el archivo, cuente las palabras y retorne el número (o 0 si el archivo no existe).
- Úsala en main.py y muestra cuántas palabras tiene cada archivo creado.

Pista: `len(contenido.split())` cuenta las palabras de un string.

Solución sugerida — Ejercicio 2.3

 **archivo_util.py** (función nueva)

```
def contar_palabras(ruta):  
    """Retorna el número de palabras del archivo o 0 si no existe."""  
    contenido = leer_archivo(ruta)  
    if contenido is None:  
        return 0  
    return len(contenido.split())
```

 **main.py** (uso)

```
from archivo_util import escribir_archivo, contar_palabras  
  
escribir_archivo('texto.txt', 'Python es un lenguaje poderoso y elegante')  
  
n = contar_palabras('texto.txt')  
print(f'El archivo tiene {n} palabras.')  
  
n2 = contar_palabras('no_existe.txt')  
print(f'Palabras en archivo inexistente: {n2}') # → 0
```

Explicación:

- Las funciones del módulo encapsulan el try/except. El programa principal (main.py) trabaja sin ver un solo try.
- Retornar None o 0 en lugar de lanzar error permite que el código que llama decida qué hacer.
- Este patrón (módulo con funciones seguras) es como se construyen librerías profesionales.

PARTE 2: EJERCICIOS INDEPENDIENTES

Practica lo aprendido sin guía — Solo enunciado y datos de prueba

Módulos

Ejercicio P1 — Módulo de conversiones

Enunciado

Crea un módulo conversiones.py con funciones para: celsius a fahrenheit, km a millas ($\text{km} \times 0.621371$), kg a libras ($\text{kg} \times 2.20462$) y metros a pies ($\text{m} \times 3.28084$). Crea main.py con un menú que importe el módulo y permita al usuario elegir qué conversión hacer.

Archivos necesarios:

```
proyecto/  
├── conversiones.py  
└── main.py
```

Ejemplo de uso	Salida esperada
Opción 1: 100°C	100°C = 212.0°F
Opción 2: 50 km	50 km = 31.07 millas

Pista: from conversiones import celsius_a_fahrenheit, km_a_millas, ... Usa un diccionario para el menú.

Ejercicio P2 — Módulo de validaciones reutilizable

Enunciado

Crea un módulo validaciones.py con funciones que retornen True/False: es_email_valido(correo), es_numero_positivo(n), es_contrasena_segura(clave) (mínimo 8 chars, una mayúscula, un número). Úsalo desde main.py para validar datos del usuario.

Archivos necesarios:

```
proyecto/  
├── validaciones.py  
└── main.py
```

Ejemplo de uso	Salida esperada
es_email_valido('a@b.com')	True

es_contrasena_segura('abc')	False — muy corta
-----------------------------	-------------------

Pista: Cada función retorna True o False. En main.py: if not validaciones.es_email_valido(correo): print('Inválido').

Ejercicio P3 — Módulo con subcarpeta: sistema escolar

Enunciado

Crea la estructura: escuela/modulos/estudiantes.py (funciones: agregar, buscar, listar) y escuela/modulos/notas.py (funciones: calcular_promedio, clasificar). En escuela/main.py importa ambos módulos y construye un menú que permita registrar estudiantes y sus notas.

Archivos necesarios:

```
escuela/  
├── main.py  
└── modulos/  
    ├── __init__.py  
    ├── estudiantes.py  
    └── notas.py
```

Ejemplo de uso	Salida esperada
Agregar: 'Ana', notas [4.5,3.8,4.0]	Ana registrada. Promedio: 4.1 — Aprobada

Pista: from modulos.estudiantes import agregar, listar
from modulos.notas import calcular_promedio, clasificar

Manejo de Errores

Ejercicio P4 — Acceso seguro a listas y diccionarios

Enunciado

Crea funciones que accedan a listas y diccionarios de forma segura con try/except. Función obtener_elemento(lista, indice) que captura IndexError. Función obtener_valor(diccionario, clave) que captura KeyError. Ambas retornan None si falla, nunca lanzan error al código que las llama.

Ejemplo de uso	Salida esperada
obtener_elemento([1,2,3], 10)	None (sin error)
obtener_valor({'a':1}, 'z')	None (sin error)

Pista: except IndexError: return None | except KeyError: return None

Ejercicio P5 — Lector de números desde archivo

Enunciado

Crea un programa que lea un archivo de texto donde cada línea es un número. Calcula la suma y el promedio. El programa debe manejar: archivo inexistente (FileNotFoundError), líneas que no son números (ValueError), y archivo vacío. Muestra un mensaje descriptivo para cada caso.

Ejemplo de uso	Salida esperada
numeros.txt con: 10,20,abc,30	Línea 3 ignorada: 'abc' no es número.Suma: 60 Promedio: 20.0

Pista: for linea in f.readlines(): try: suma += float(linea.strip()) except ValueError: print(...)

Ejercicio P6 — Decorador de seguridad (avanzado)

Enunciado

Crea una función ejecutar_seguro(funcion, *args) que recibe cualquier función y sus argumentos, la ejecuta dentro de un try/except y retorna el resultado o un mensaje de error. Úsala para envolver funciones que pueden fallar sin modificar el código de esas funciones.

Ejemplo de uso	Salida esperada
ejecutar_seguro(int, 'abc')	Error: invalid literal for int()...
ejecutar_seguro(lambda a,b: a/b, 10, 2)	5.0

Pista: try: return funcion(*args) | except Exception as e: return f'Error: {e}'

Módulos + Errores (Combinados)

Ejercicio P7 — Sistema de registro con módulos y validación

Enunciado

Construye un sistema con tres módulos: validaciones.py (valida datos con try/except y raise), registro.py (guarda y carga datos en JSON con manejo de errores), y main.py (menú que los une). Registra usuarios con nombre, edad y correo. Los datos persisten entre ejecuciones.

Archivos necesarios:

```
sistema/  
├── main.py  
├── validaciones.py  
└── registro.py
```

Ejemplo de uso	Salida esperada
Registrar: Ana, 25, ana@mail.com	✓ Usuario registrado
Registrar: Ana, abc, @	⚠ Edad inválida ⚠ Correo sin dominio

Pista: En registro.py usa try/except FileNotFoundError al cargar JSON. En validaciones.py usa raise ValueError para cada regla que falle.

Ejercicio Final — Sistema Modular con Manejo Total de Errores

Enunciado

Construye una aplicación completa que integre todo lo visto en el tema.

Estructura requerida:

```
inventario_app/  
├── main.py           ← menú principal  
├── modulos/  
│   ├── __init__.py  
│   ├── validaciones.py ← valida entradas del usuario  
│   ├── productos.py   ← lógica de negocio  
│   └── persistencia.py ← lee/escrbe JSON con try/except
```

El sistema debe hacer:

1. Ver inventario completo (carga desde JSON, maneja FileNotFoundError).
2. Agregar producto: valida nombre (no vacío), precio (número positivo), stock (entero ≥ 0).
3. Registrar venta: verifica stock disponible, lanza un ValueError personalizado si no hay suficiente.
4. Ver reporte: muestra total de productos, valor total en bodega y producto con mayor stock.
5. Guardar y salir: guarda el inventario en JSON con manejo de PermissionError.

Restricciones:

- Ninguna función en productos.py usa input() — todo viene como parámetros.
- Ningún módulo importa a otro módulo del mismo proyecto (solo main.py importa todo).
- Todos los errores de archivo se manejan en persistencia.py — nunca se propagan a main.py.
- Todas las validaciones de usuario se manejan en validaciones.py con raise y try/except.

¡Felicidades por completar los ejercicios!

"Divide tu código en módulos y maneja los errores con elegancia"

Ahora puedes organizar tu código y manejar errores como un profesional.

Módulos para organizar. try/except para controlar. iter/next para recorrer con precisión.